

Prototyp Skizze „Tausendfüßler“

Beschreibung

„Tausendfüßler“ ist ein verteilter Webcrawler, der über einen Telegram-Bot gesteuert wird. Ein Bot-Prozess nimmt Crawl-Aufträge entgegen und reicht sie über eine Socket-Verbindung an einen Koordinator weiter. Der Koordinator verteilt die zu besuchenden URLs auf mehrere Worker-Prozesse, die parallel Webseiten herunterladen, parsen und ihre Ergebnisse an den Koordinator zurückmelden. Der Koordinator streamt die gefundenen Seiten in Echtzeit über den Bot an den Nutzer zurück.

Funktionale Anforderungen

Benutzerinteraktion (via Telegram-Bot)

- Nutzer können einen Crawl-Auftrag mit Start-URL, maximaler Tiefe und optionalen Filtern anlegen.
- Nutzer können alle ihre aktiven und abgeschlossenen Aufträge mit Kurzinfo auflisten.
- Nutzer können zu einem Auftrag Details abrufen: aktuelle Tiefe, Anzahl besuchter Seiten, Status (laufend/pausiert/beendet/abgebrochen).
- Nutzer können einen laufenden Auftrag pausieren, fortsetzen oder abbrechen. Bei Abbruch bleiben bereits gesammelte Ergebnisse erhalten.
- Während eines Crawls erhalten Nutzer kontinuierlich neu gefundene Seiten als Live-Stream (URL, Titel, Textanfang) in den Telegram-Chat.
- Nach Abschluss eines Auftrags wird ein zusammenfassender Report mit Anzahl besuchter Seiten, extrahierter Links, Fehlern und Gesamtdauer gesendet.

Systeminterne Abläufe (Koordinator & Worker)

- Der Koordinator nimmt Aufträge vom Bot über eine TCP-Socket-Verbindung entgegen und verwaltet sie in einer Job-Queue.
- Der Koordinator verteilt zu crawlende URLs entweder per Round-Robin oder lastabhängig (Least-Work-First) an registrierte Worker: Der Worker mit der kürzesten aktuellen Warteschlange erhält das nächste URL-Paket.
- Worker holen sich aktiv URL-Pakete vom Koordinator ab und melden gecrawlte Seiten sowie neu gefundene Links zurück.
- Der Koordinator dedupliziert URLs pro Auftrag thread-safe und über Worker-Grenzen hinweg, sodass jede URL nur einmal besucht wird.
- Worker setzen die vom Koordinator vorgegebenen Tiefenstufen um und respektieren Pausierungs- und Abbruchsignale. Koordinator pusht Pausierungs-/Abbruchsignale als JSON über TCP; Worker prüft dies nach jeder Seite, beendet laufende Abrufe und wartet ggf. in einer Schleife auf ein Resume-Signal.
- Bei einem Worker-Absturz oder unerwarteten Verbindungsabbruch erkennt der Koordinator den geschlossenen Socket, markiert alle unfertigen URLs wieder als „offen“, protokolliert den Vorfall und verteilt sie datenverlustfrei neu.
- Der Koordinator schreibt alle relevanten Ereignisse (Auftrag gestartet/beendet, Worker-Fehler, etc.) in eine Log-Datei.
- Crawl-Ergebnisse werden als JSON-Dateien im Dateisystem abgelegt und können für historische Reports erneut eingesehen werden.

Wartung & optionale Erweiterungen

- Das System berechnet auf Anfrage einfache Nutzungsstatistiken (Anzahl Aufträge, meist gecrawlte Domains, Gesamtzahl besuchter Seiten) aus den vorliegenden Ergebnisdateien.
- Crawl-Ergebnisse, die älter als eine konfigurierbare Anzahl Tage sind, können beim Systemstart automatisch gelöscht werden (Standard: 30 Tage).

Nicht-funktionale Anforderungen

Bei diesen Anforderungen wird davon ausgegangen, dass Koordinator und Worker auf einem handelsüblichen Laptop (mehrkernige CPU) laufen.

- Der Koordinator beantwortet korrekte Status- und Steueranfragen des Bots zu >99,9 % ohne internen Fehler.
- Eine Statusabfrage wird bei einer Last von <20 gleichzeitigen Aufträgen in unter 0,2 s beantwortet.
- Das System (Koordinator und Worker) nutzt auf mehrkernigen Systemen dynamisch Multithreading, indem Worker mit einem Thread-Pool arbeiten, dessen Größe sich an die verfügbaren Kerne anpasst.
- URL-Deduplizierung im Koordinator sowie die gleichzeitige Entgegennahme von Ergebnissen mehrerer Worker sind atomar und thread-safe; es treten keine Doppel-Crawls oder Lost-Updates auf.
- Live-Ergebnisse werden innerhalb von 2 Sekunden nach Abschluss des Seitenabrufs durch einen Worker über den Koordinator an den Bot ausgeliefert.
- Der Crawl-Durchsatz (Seiten pro Sekunde) mit zwei Workern ist um mindestens 60 % höher als mit einem Worker – unter Verwendung desselben Test-Crawl-Auftrags.
- Der Koordinator ist innerhalb von 15 Sekunden nach Start bereit, Socket-Verbindungen von Bot und Workern anzunehmen (Programm bereits gebaut).
- Der Worker-Threadpool liefert unter gleichzeitiger Last garantiert alle Ergebnisse vollständig zurück; kein Task darf durch Race Conditions oder fehlerhafte Future-Verkettung verloren gehen.
- Security bleibt als nicht-funktionale Anforderung explizit ausgeklammert, um die Prototypprüfung einfach zu halten.

Verteilung

Nebenläufigkeit:

- Multithreading pro Worker: Paralleles Herunterladen und Parsen mehrerer Seiten mit einem konfigurierbaren Thread-Pool.
- Parallele Verarbeitung eingehender Worker-Ergebnisse im Koordinator (separate Threads für jede Worker-Verbindung und für das Weiterleiten an den Bot).
- Parallele Statistikberechnung durch Einlesen und Aggregieren der JSON-Ergebnisdateien.

Verteilung:

- Bot ↔ Koordinator: TCP-Socket-Verbindung (textbasiertes JSON-Protokoll), Bot sendet Aufträge und Steuerbefehle, Koordinator streamt Ergebnisse und Status.

- Koordinator ↔ Worker: TCP-Socket-Verbindungen (pro Worker eine persistente Leitung), über die URL-Pakete zugewiesen und Ergebnisse zurückgesendet werden.
- Worker sind eigenständige Java-Prozesse, die theoretisch auf separaten Rechnern ausgeführt werden können.

Komponenten:

- **Bot und Koordinator** sind als Spring-Boot-Anwendungen ohne Web-MVC realisiert; Dependency Injection und Service-Klassen bilden die interne Struktur.
- **Worker** sind als schlanke Plain-Java-Prozesse implementiert (kein Spring Boot). Der Worker hat eine klar begrenzte Aufgabe: URLs crawlen und Ergebnisse über die bestehende TCP-Verbindung zurücksenden. Spring Boot würde hier nur unnötigen Start-Overhead und Abhängigkeiten einbringen, ohne praktischen Mehrwert. Die gesamte Kommunikation mit dem Koordinator läuft ausschliesslich über den TCP-Socket (line-delimited JSON) – ein zweiter HTTP-Server auf dem Worker ist nicht vorgesehen und wäre architektonisch falsch.
- Das System stellt keine eigene GUI und kein eigenes REST-API nach außen bereit. Die einzige externe Schnittstelle ist der Telegram-Bot, der intern die offizielle Telegram-HTTP-API als Kommunikationskanal nutzt – dieser HTTP-Verkehr ist kein Teil der Systemarchitektur, sondern ein externer Dienst.

Skizze

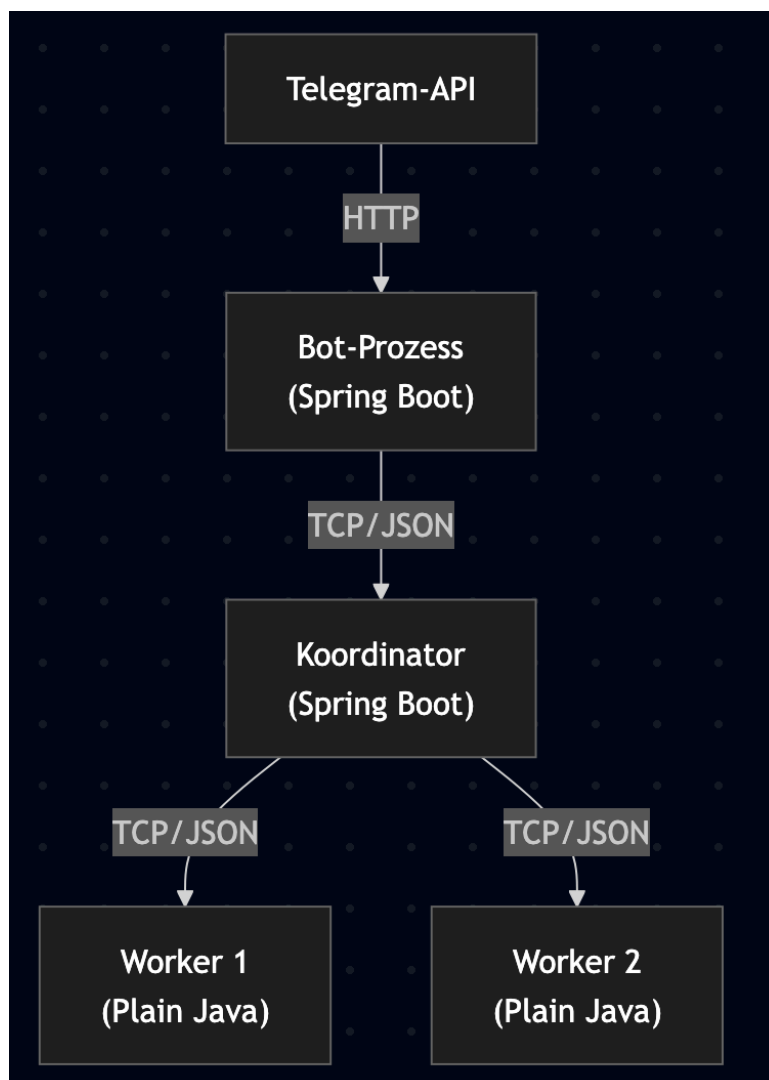


Abbildung 1: Systemarchitektur – Telegram-API → Bot → Koordinator → Worker

Last-Simulation & Test der nicht-funktionalen Anforderungen

Nicht-funktionale Anforderung	Überprüfung
Koordinator beantwortet korrekte Statusanfragen zu >99,9% ohne internen Fehler	Ein Testclient sendet über einen Zeitraum von mehreren Minuten eine definierte Menge gültiger Statusabfragen und ungültiger Requests. Der Koordinator loggt interne Fehler getrennt von Clientfehlern. Aus dem Log wird das Verhältnis von internen Fehlern zu Gesamtanfragen berechnet.
Statusabfragen unter 0,2 s bei <20 gleichzeitigen Aufträgen	Der Testclient legt zuerst 20 Crawl-Jobs an und feuert dann getaktet Statusabfragen ab. Für jede Anfrage wird die Antwortzeit gemessen. Überschreitungen werden gezählt.
Dynamische Mehrkernauslastung (Worker und Koordinator)	Crawl-Threads und Worker-Handler loggen ihre Thread-IDs. Nach einem Crawl-Durchlauf wird die Varianz der Aufgaben pro Thread ermittelt. Eine gleichmäßige Verteilung (Standardabweichung < 20 % vom Mittelwert) gilt als Nachweis.
Atomare und thread-sichere URL-Deduplizierung	Ein Testclient injiziert parallel 1000 identische URLs über zwei Worker in denselben Auftrag. Nach Abschluss wird die Ergebnisliste abgefragt. Ist eine URL mehrfach enthalten oder tauchen verlorene Updates auf, ist die NFA nicht erfüllt.
Live-Ergebnisse innerhalb 2s nach Seitenabruf	Der Testclient überwacht einen dedizierten Crawl-Auftrag und speichert die Zeitstempel, sobald ein Worker ein Ergebnis absendet (Worker-seitig protokolliert) und sobald es beim Bot ankommt. Die Differenz wird gemessen; sie darf 2 s nicht überschreiten.
≥60 % Durchsatzsteigerung mit 2 Workern vs. 1 Worker	Ein fester Crawl-Auftrag (gleiche Start-URL, gleiche maximale Seitenzahl) wird zuerst mit einem, dann mit zwei Workern durchgeführt. Der Koordinator loggt Start-/Endzeit und Anzahl verarbeiteter Seiten. Daraus wird der Durchsatz berechnet und der relative Anstieg bestimmt.
Worker-Threadpool liefert alle Ergebnisse unter Last	Ein Testclient submittiert 100 identische URLs gleichzeitig an einen CrawlExecutor mit 4 Threads. Nach Abschluss wird die Anzahl der zurückgegebenen CrawlOutcome-Objekte geprüft. Sie muss exakt 100 betragen – kein Ergebnis darf durch Race Conditions oder fehlerhafte Future-Verkettung verloren gehen.
Koordinator in <15 s startbereit	Die Spring-Startup-Logs des Koordinators werden analysiert. Zusätzlich sendet der Testclient unmittelbar nach Prozessstart einen Dummy-Befehl über den TCP-Socket und misst die Zeit bis zur ersten erfolgreichen Antwort. Worker-Prozesse (Plain Java) haben keinen Spring-Startup-Overhead und gelten als sofort bereit sobald die Socket-Verbindung zum Koordinator aufgebaut ist.
Parallele Statistikberechnung aus JSON-Ergebnisdateien	Es werden mindestens 50 JSON-Ergebnisdateien im Verzeichnis hinterlegt und der /stats-Befehl aufgerufen. Der Datei-Parser loggt die beteiligten Thread-IDs. Der Test verifiziert, dass die Dateien von mehr als einer eindeutigen Thread-ID parallel verarbeitet wurden, und die Korrektheit der aggregierten Zahlen wird anhand von Referenzwerten geprüft.

